

Lab 10: NEW API Topics

Aim: To study and practice various updates introduced since Java 8 and onward.

Programming Exercise:

Program set 1:

1. Create an interface Greeter with a default method greet() that prints "Hello from the interface!". Implement this interface in a class and invoke the method.

ANS:-

CODE:-

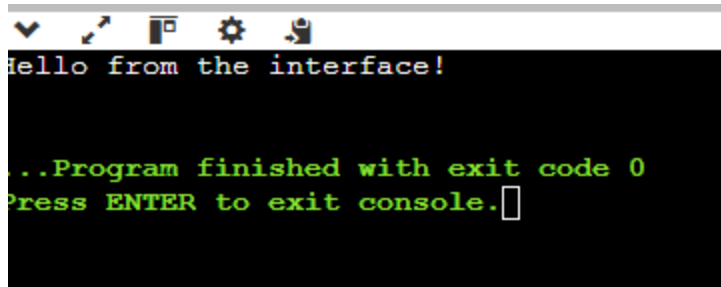
```
import java.util.*;

interface Myinterface{
    default void show(){
        System.err.println( "Hello from the interface!");
    }
}

class Test implements Myinterface{
    public void show(){
        Myinterface.super.show();
    }
}

public class main {
    public static void main(String[] args)
    {
        Myinterface obj = new Test();
        obj.show();
    }
}
```

OUTPUT:-



```
Hello from the interface!

...Program finished with exit code 0
Press ENTER to exit console.
```

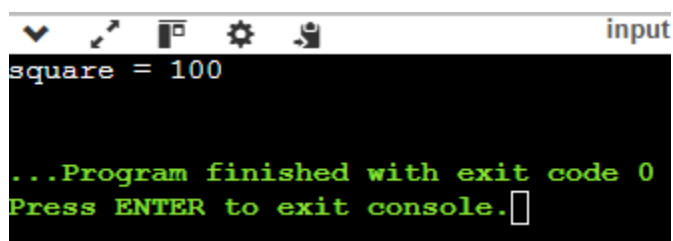
2. Create an interface MathUtil with a static method square(int n) that returns the square of a number. Call this method from main().

ANS:-

CODE:-

```
import java.util.*;
interface MathUtil{
    int square(int a);
}
public class main {
    public static void main(String[] args)
    {
        MathUtil obj = (n) -> n*n;
        System.out.println("square = " + obj.square(10));
    }
}
```

OUTPUT:-



```
square = 100

...Program finished with exit code 0
Press ENTER to exit console.
```

3. Write a lambda expression to create a Runnable that prints "Thread running" and executes it.

ANS:-

CODE:-

```
import java.util.*;

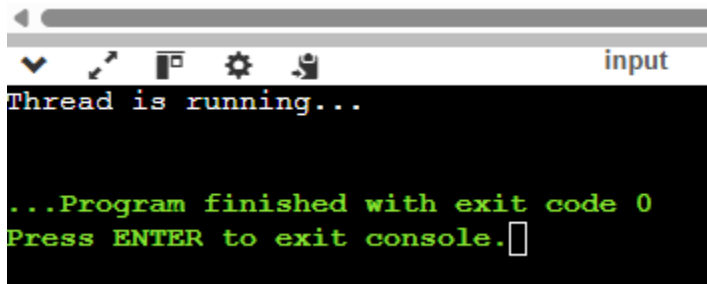
public class main {

    public static void main(String[] args){

        Runnable r = () -> System.out.println("Thread is running...");

        Thread t = new Thread(r);
        t.start();
    }
}
```

OUTPUT:-



4. Write a Java program to create a functional interface MaxFinder with a method max(int a, int b). Use a lambda expression to find and display the maximum of two numbers.

ANS:-

CODE:-

```
import java.util.*;

interface MaxFinder {

    int max(int a, int b);

}

public class main {
```

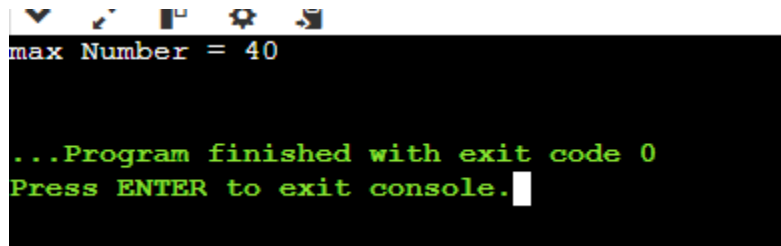
```

public static void main(String[] args)
{
    MaxFinder obj = (a,b) ->{
        if(a>b){
            return a;
        }
        else{
            return b;
        }
    };
    System.out.println("max Number = " + obj.max(40,20));

}
}

```

OUTPUT:-



```

max Number = 40

...Program finished with exit code 0
Press ENTER to exit console.

```

- Given a list of integers, use Stream API to filter and print only the odd numbers.

ANS:-

CODE:-

```

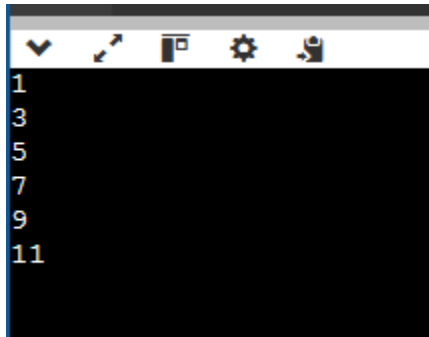
import java.util.*;

public class main {
    public static void main(String[] args)
    {
        List<Integer> list = Arrays.asList(1,2,3,4,5,6,7,8,9,10,11,12);
        list.stream()
            .filter(n -> n%2 !=0)

```

```
        .forEach(System.out::println);
    }
}
```

OUTPUT:-



6. Given a list of strings, use Stream API to convert each string to uppercase and print the result.

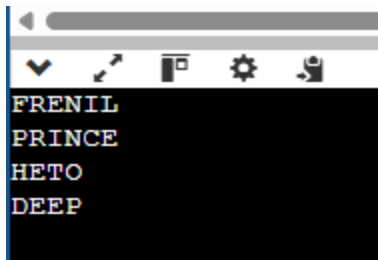
ANS:-

CODE:-

```
import java.util.*;

public class main {
    public static void main(String[] args)
    {
        List<String> list = Arrays.asList("frenil","prince","heto","deep");
        list.stream()
            .map(n ->n.toUpperCase())
            .forEach(System.out::println);
    }
}
```

OUTPUT:-



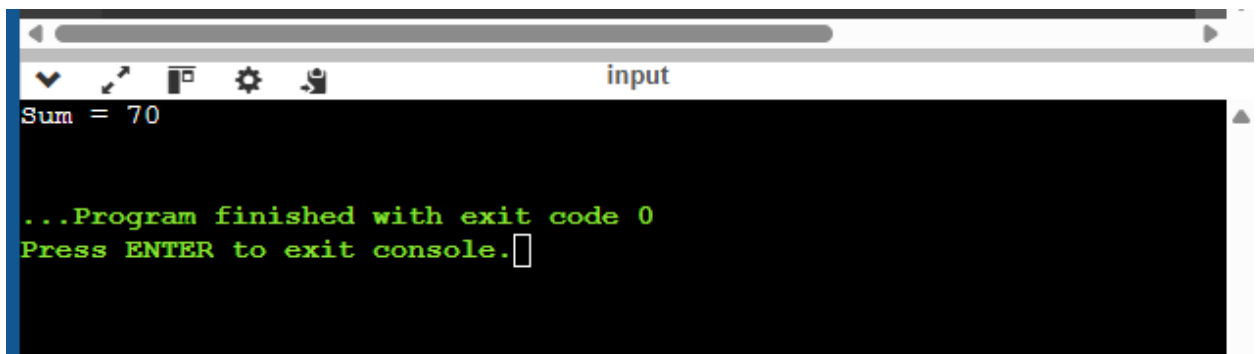
7. Given a list of integers, use Stream API `reduce()` to find the sum of all elements.

ANS:-

CODE:-

```
import java.util.*;  
public class main {  
    public static void main(String[] args) {  
        List<Integer> list = Arrays.asList(20, 20, 30);  
        int sum = list.stream()  
            .reduce (0, (a, b) -> a + b);  
        System.out.println("Sum = " + sum);  
    }  
}
```

OUTPUT:-



8. Given a list of names, use Stream API to filter names starting with letter 'A' and collect them into a new list.

ANS:-

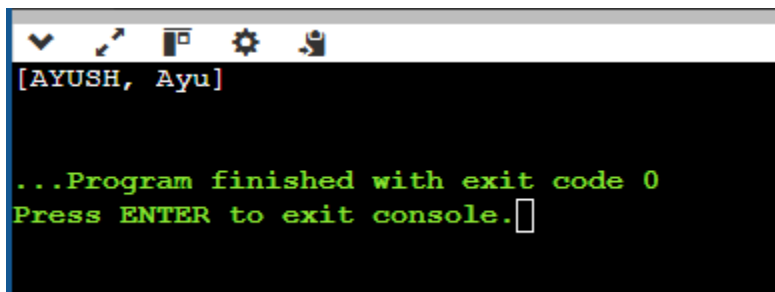
CODE:-

```
import java.util.*;
import java.util.stream.Collectors;

public class main {
    public static void main(String[] args)
    {
        List<String> list = Arrays.asList("frenil", "AYUSH", "ayush", "deep", "Harsid",
        "heto","Ayu");
        List<String> newlist = list.stream()
            .filter(s -> s.startsWith("A"))
            .collect(Collectors.toList());
        System.out.println(newlist);

    }
}
```

OUTPUT:-

A screenshot of a Java IDE's console window. The window has a title bar with standard OS icons. The console output is displayed on a black background with green text. The first line of output is "[AYUSH, Ayu]". The second line is "...Program finished with exit code 0". The third line is "Press ENTER to exit console." followed by a small white cursor icon.

9. Create an Optional<String> with a value. If the value is present, print it using ifPresent().

ANS:-

CODE:-

```
import java.util.*;
import java.util.stream.Collectors;
```

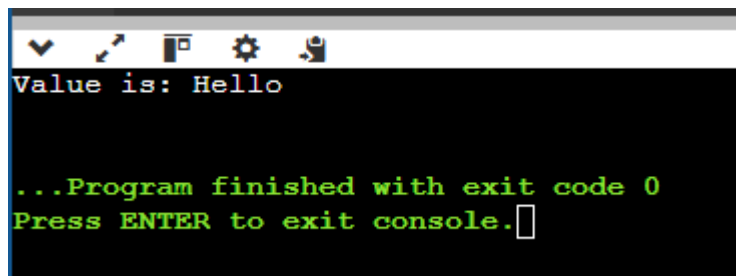
```

public class main {
    public static void main(String[] args)
    {
        Optional<String> a = Optional.of("Hello");
        a.ifPresent(value -> System.out.println("Value is: " + value));

    }
}

```

OUTPUT:-



The screenshot shows a Java IDE console window. The output text is: "Value is: Hello" in red font. Below it, in green font, it says "...Program finished with exit code 0" and "Press ENTER to exit console." followed by a cursor icon.

10. Create an immutable list using List.of() and attempt to add a new element. What is the outcome?

ANS:-

CODE:-

```

import java.util.*;
import java.util.stream.Collectors;

public class main {
    public static void main(String[] args)
    {
        List<Integer> list = Arrays.asList(10, 20, 30, 40);
        list.add(50);

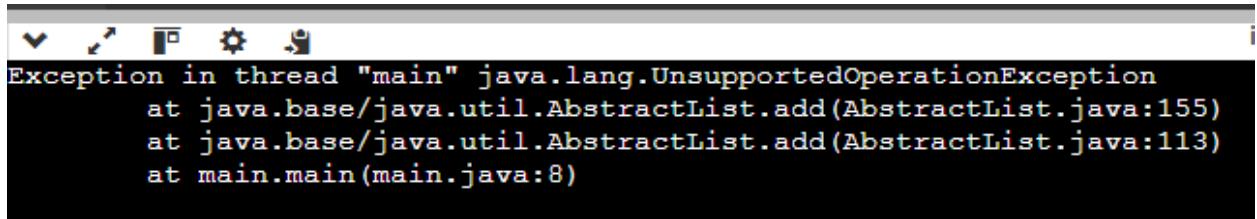
    }
}

```



```
}
```

OUTPUT:-

A screenshot of a Java IDE's console window. The window has a title bar with standard OS icons. The console text shows an exception in the main thread: java.lang.UnsupportedOperationException. The stack trace points to the add method of AbstractList in the java.base package, specifically at lines 155 and 113, and then to the main method at line 8 of main.java.

```
Exception in thread "main" java.lang.UnsupportedOperationException
    at java.base/java.util.AbstractList.add(AbstractList.java:155)
    at java.base/java.util.AbstractList.add(AbstractList.java:113)
    at main.main(main.java:8)
```

11. Create an immutable set using Set.of() and print all elements using forEach() with a method reference.

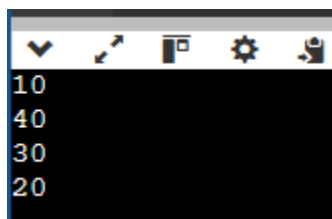
ANS:-

CODE:-

```
import java.util.*;
import java.util.stream.Collectors;

public class main {
    public static void main(String[] args)
    {
        Set<Integer> list = Set.of(10, 20, 30, 40);
        list.forEach(System.out::println);
    }
}
```

OUTPUT:-

A screenshot of a Java IDE's console window. The window has a title bar with standard OS icons. The console text shows the output of the program: the numbers 10, 40, 30, and 20, each on a new line.

```
10
40
30
20
```

Program 2:

Design a Java program to process student results in a university portal system.

Each student record contains: name, marks, passed status

The system should:

1. Filter students who have passed and scored more than 60 marks.
2. Convert each filtered student into a formatted message.
3. Display the messages on the console.

Tasks to Perform:

- Create a Student class with required fields.
- Store at least 5 student objects in a list.
- Use:
 - Predicate<Student> to filter students
 - Function<Student, String> to format output
 - Consumer<String> to display results
- Use lambda expressions (no traditional filtering loops).
- Use method reference for printing (if applicable).

ANS:-

CODE:-**Ans:**

```
import java.util.ArrayList;
import java.util.List;
import java.util.function.Consumer;
import java.util.function.Function;
import java.util.function.Predicate;
class Student {
    private String name;
    private int marks;
    private boolean passed;
    public Student(String name, int marks, boolean passed) {
        this.name = name;
        this.marks = marks;
        this.passed = passed;
    }
    public String getName() { return name; }
```

```

    public int getMarks() { return marks; }
    public boolean isPassed() { return passed; }
}
public class student {
    public static void main(String[] args) {

        List<Student> students = new ArrayList<>();
        students.add(new Student("prince", 85, true));
        students.add(new Student("deep", 55, true));
        students.add(new Student("frenil", 90, true));
        students.add(new Student("harshid", 70, false));
        students.add(new Student("hetex", 62, true));
        students.add(new Student("sujal", 40, false));

        Predicate<Student> isEligible = s -> s.isPassed() && s.getMarks() > 60;

        Function<Student, String> formatMessage = s ->
            "Student: " + s.getName() + " | Marks: " + s.getMarks() + " | Status: Qualified";

        Consumer<String> display = System.out::println;
        System.out.println("--- Eligible Students (Passed & > 60 Marks) ---");

        students.stream()
            .filter(isEligible)
            .map(formatMessage)
            .forEach(display);
    }
}

```

OUTPUT:-

```

--- Eligible Students (Passed & > 60 Marks) ---
Student: prince | Marks: 85 | Status: Qualified
Student: frenil | Marks: 90 | Status: Qualified
Student: hetex | Marks: 62 | Status: Qualified

```